Carlos
Introdução

# Vesta

Sistema de gestão hoteleira: uma API REST para reservas, hóspedes, quartos e funcionários.

Projeto Interdisciplinar, Desenvolvimento de Software Multiplataforma, Fatec Mauá
Thiago R. Alves, George, Augusto e Carlos

Carlos
Introdução

# Um hotel não pode vender o mesmo quarto duas vezes

## O problema

- Reservas que se sobrepõem no mesmo quarto
- Hóspede colocado num quarto em manutenção ou pequeno demais
- Dados de hóspedes, quartos e equipe espalhados

## O objetivo

Centralizar a operação do hotel numa API que guarda tudo num só lugar e *recusa* os erros antes que eles cheguem ao banco.

# O que usamos

| Tecnologia | Para quê |
|---|---|
| **Java 17 + Spring Boot 4.1** | Base da aplicação; sobe o servidor em `localhost:8080` |
| **Spring Web MVC** | Os endpoints REST em `/api/...` |
| **Spring Data JPA** | Entidades viram tabelas; consultas sem SQL escrito à mão |
| **H2 em memória** | Banco recriado com dados de exemplo a cada execução |
| **Bean Validation** | Regras de preenchimento direto nos modelos |
| **Swagger (springdoc 3.0.3)** | Documentação e testes da API pelo navegador |
| **Trello** | Gestão ágil do projeto em quadro Kanban |

# Como o sistema está organizado

Carlos
Introdução

Arquitetura em camadas (MVC), pacote `br.com.vesta`. Cada requisição atravessa as camadas nesta ordem:

`controller`

### Recebe

Um controller REST por entidade. Valida o JSON e responde com o código HTTP certo.

›

`service`

### Decide

`AlocacaoService` aplica as regras de alocação de quarto.

›

`repository`

### Busca

Interfaces `JpaRepository` com consultas derivadas do nome do método.

›

`model`

### Guarda

Entidades JPA que viram as tabelas do H2.

Por fora de tudo, o `GlobalExceptionHandler` padroniza qualquer erro numa resposta JSON.

# Quem fez cada parte

## Thiago

Entidades JPA, relacionamentos e o banco H2 que se recria já populado.

## George

Controllers REST, filtros de pesquisa e a integração com o Swagger.

## Augusto

Quadro no Trello e gestão ágil do projeto.

## Carlos

Apresentação: introdução e encerramento.

A seguir, cada um apresenta a própria parte. Começando pela base: os dados.

# Cinco entidades, três relações

**Thiago**
Entidades e banco



# As regras moram na entidade

**Thiago**
Entidades e banco

```java
@Entity
@Table(name = "hospedes")
public class Hospede {

  @NotBlank(message = "O CPF é obrigatório")
  @Pattern(regexp = "\\d{11}")
  private String cpf;

  @NotNull @Past
  private LocalDate nascimento;

  @ManyToOne
  @JoinColumn(name = "reserva_id")
  private Reserva reserva;

  @ManyToOne
  @JoinColumn(name = "dependencia_id")
  private Dependencia quarto;
}
```

- **Bean Validation** nos campos: `@NotBlank`, `@Pattern`, `@Email`, `@Past`, `@Positive`
- Mensagens de erro em português, prontas para o usuário
- **Relações N:1** com `@ManyToOne`: hóspede → reserva e quarto; dependente → hóspede
- Status do quarto como enum `StatusDependencia`, gravado como texto: `DISPONIVEL`, `OCUPADO`, `MANUTENCAO`

# Um banco que nasce pronto toda vez

**Thiago**
Entidades e banco

```
# application.properties
spring.datasource.url=
  jdbc:h2:mem:vesta
spring.jpa.hibernate.ddl-auto=create
spring.h2.console.enabled=true
```

- `ddl-auto=create` apaga e recria as tabelas a cada execução
- O `import.sql` roda logo depois e popula tudo, na ordem certa das chaves estrangeiras
- Resultado: todo teste e toda demonstração começam do mesmo estado

| 4 | 7 | 5 | 5 | 4 |
|---|---|---|---|---|
| funcionários | dependências | reservas | hóspedes | dependentes |

Conferência dos dados no console do H2: `/h2-console`

# Um controller para cada entidade

**George**
Controllers e Swagger

| Verbo | Rota | Ação |
|---|---|---|
| `POST` | `/api/hospedes` | Cria |
| `GET` | `/api/hospedes` | Lista |
| `GET` | `/api/hospedes/{id}` | Busca um |
| `PUT` | `/api/hospedes/{id}` | Edita |
| `DELETE` | `/api/hospedes/{id}` | Exclui |

O mesmo CRUD completo existe para `/reservas`, `/hospedes`, `/dependentes`, `/dependencias` e `/funcionarios`.

## Respostas

**200** OK · **201** criado · **204** excluído · **400** dados inválidos · **404** não encontrado · **409** conflito

`@Valid` no corpo da requisição dispara as validações das entidades.

# Filtros sem escrever SQL

**George**
Controllers e Swagger

No Spring Data JPA, o nome do método vira a consulta:

```
interface ReservaRepository
    extends JpaRepository<Reserva, Long> {

  List<Reserva> findByCheckinBetween(
      LocalDate inicio, LocalDate fim);

  List<Reserva> findByCpf(String cpf);
}
```

| Entidade | Filtros em `/filtro/...` |
|---|---|
| **Reservas** | checkin, período, pagamento, CPF, tipo de quarto, ativas |
| **Dependências** | status, capacidade, tipo, acessibilidade, disponíveis no período |
| **Hóspedes** | reserva, quarto, CPF, nome |
| **Dependentes** | hóspede, nome |
| **Funcionários** | departamento, cargo, nível de acesso, nome |

# O que a API recusa

**George**
Controllers e Swagger

- Checkout antes do check-in
- Hóspede em área comum ou em quarto em manutenção
- Reserva com mais pessoas que a capacidade do quarto
- Duas reservas com datas sobrepostas no mesmo quarto
- Excluir reserva ou quarto que ainda tem hóspedes
- CPF de funcionário repetido

Regra quebrada vira resposta clara, em vez de erro genérico:

```
HTTP 409 Conflict
{
  "erro": "O quarto já está reservado
           nesse período"
}
```

Campos inválidos voltam como **400** com a lista de campos e mensagens; JSON malformado e datas fora de `AAAA-MM-DD` também.

# Swagger: a API testada pelo navegador

- Dependência `springdoc-openapi` 3.0.3, a série compatível com Spring Boot 4
- Nenhuma configuração extra: ele lê controllers e modelos e gera a documentação
- Interface em `/swagger-ui/index.html`
- Especificação OpenAPI em `/v3/api-docs`

Hóspedes e dependentes referenciam os “pais” só pelo id: `"reserva": {"id": 1}`

## Demonstração

1. Listar os quartos disponíveis no período
2. Criar uma reserva nova
3. Alocar um hóspede no quarto
4. Tentar a mesma data de novo e ver o **409**

# Kanban no Trello

Augusto
Gestão ágil

Cada cartão anda da esquerda para a direita:

1. Backlog
2. A Fazer
3. Em Andamento
4. Em Teste/Validação
5. Concluído

Nenhum cartão chega a Concluído sem antes passar pela coluna de validação.

## Projeto Ágil: Sistema de hotelaria Vesta

O quadro completo, com todas as histórias, critérios de aceitação e checklists

`trello.com/b/6accf61b1f001db4400cca6a`

**Abrir o quadro no Trello**

# Quatro frentes, uma cor para cada

**Augusto**
Gestão ágil

**Gestão de Reservas e Hóspedes**

Cadastrar hóspede, editar e cancelar reserva, buscar reservas, criar reserva pelo calendário

**Controle Operacional de Quartos**

Registrar quarto em manutenção, atualizar status após a limpeza

**Faturamento e Painel Gerencial**

Consumos extras, resumo financeiro, taxa de ocupação, faturamento por período, relatórios

**Adequação às novas tecnologias**

H2 e dados de exemplo, testes no Swagger, filtros e erros, autenticação, DTOs, testes automatizados

As três primeiras são funcionalidades do hotel. A vermelha reúne o trabalho técnico que sustenta tudo.

Augusto
Gestão ágil

# As histórias partem de quem usa o hotel

### Recepcionista

Cadastrar hóspede, buscar e editar reservas, reservar pelo calendário, check-in, check-out, lançar consumos e ver o resumo da conta

### Gerente

Acompanhar a taxa de ocupação, consultar o faturamento por período e gerar relatórios para a diretoria

### Chefe de governança

Colocar quarto em manutenção com o motivo e liberá-lo de novo após o reparo e a limpeza

Exemplo do cartão “Criar reserva pelo calendário visual”: como recepcionista, quero criar uma reserva num calendário de disponibilidade, para evitar overbooking e conflito de datas.




# Anatomia de um cartão

1. **Etiqueta**: a frente de trabalho do cartão
2. **História de usuário**: “Como recepcionista, quero..., para...”. Quem pede, o quê e por quê
3. **Critérios de aceitação** no formato Dado / Quando / Então, incluindo o caso de erro (CPF duplicado)
4. **Checklist com datas**: cada critério vira um item marcado quando testado. 100% é a definição de pronto

# O que aprendemos no caminho

**Carlos**
Encerramento

### Versões importam

Spring Boot 4 só funciona com a série 3.x do springdoc. Uma linha no `pom.xml` decide se o Swagger abre.

### Regras cruzam tabelas

Evitar conflito de datas exige olhar reservas, quartos e hóspedes ao mesmo tempo.

### Dividir sem se perder

Com o quadro no Trello, cada um sabia o que estava em teste, em andamento e pronto.

# Obrigado. Perguntas?

Thiago R. Alves, George, Augusto e Carlos

Projeto Interdisciplinar Vesta, Fatec Mauá